iOS SDK Introduction

WeDoBooks SDK Integrator Guide

Setup and Authentication

Goal

Initialize SDK safely, choose backend mode explicitly, and authenticate users before book operations.

Integrator Steps

1. Add SPM dependency from the SDK umbrella repo.
2. Provide a `FirebaseAdapterFactoryType` implementation from FirebaseProvider.
3. Call `WeDoBooksFacade.shared.setup(...)` once per app launch on main actor.
4. Fetch a custom token from your backend and sign in using `userOperations.signIn(with:)`.

Setup Example (Default Firebase App)

```
import WeDoBooksSDK
import FirebaseProvider

@MainActor
func configureSDK() throws {
    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: nil
    )
}
```

Setup Example (Named Firebase App)

Use a custom plist when the SDK should use a non-default Firebase app.

```
import WeDoBooksSDK
import FirebaseProvider

enum SDKSetupExampleError: Error {
    case missingFirebaseConfigFile(String)
}

@MainActor
func configureSDKWithNamedFirebasePlist() throws {
    let plistName = "GoogleService-Info-WDBSDK"
```

```
    guard let configPath = Bundle.main.path(forResource: plistName, ofType:
"plist") else {
        throw SDKSetupExampleError.missingFirebaseConfigFile(plistName)
    }

    try WeDoBooksFacade.shared.setup(
        readerKey: "<reader-key>",
        readerSecret: "<reader-secret>",
        mode: .streaming,
        firebaseAdapterFactory: FirebaseAdapterFactory(),
        firebaseConfigFilePath: configPath
    )
}
```

Token Acquisition Guidance

- Production: Host app requests token from your backend. Your backend performs backend-to-backend token exchange with the WeDoBooks token endpoint and returns a short-lived custom token to the host app.
- Testing: You can expose a demo token endpoint pattern (for example `https://example-integrator.test/wedobooks/demo-token`) that returns a token for sandbox testing flows.

Security note: Demo token endpoints are for testing and non-production flows only.

Auth Example (Sign In With Custom Token)

```
func signIn(token: String) async {
    let result = await WeDoBooksFacade.shared.userOperations.signIn(with: token)
    if case .failure(let error) = result {
        print("Sign in failed: \(error)")
    }
}
```

Flow Diagram

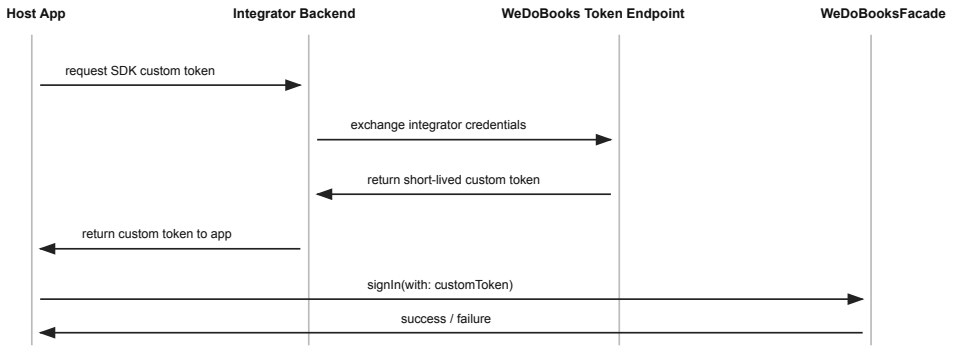

Figure 1: Setup and auth sequence

Optional Demo Token Fetch Example (Testing)

```
import Foundation

struct DemoTokenResponse: Decodable {
    let token: String
}

func fetchDemoTokenForTesting(userId: String) async throws -> String {
                                            let           encodedUser           =
userId.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? userId
   let url = URL(string: "https://example-integrator.test/wedobooks/demo-token?
user_id=\(encodedUser)")!
    let (data, _) = try await URLSession.shared.data(from: url)
    return try JSONDecoder().decode(DemoTokenResponse.self, from: data).token
}
```

Notes

- `setup(...)` is intentionally explicit about backend mode.
- Repeated setup attempts throw `SetupError.setupAlreadyCalled`.
- Most namespaces that interact with user data require a signed-in user.

Cross References

- DocC: GettingStarted.md
- DocC symbol root: `WeDoBooksFacade`

Checkout and Open

Goal

Request/observe user checkouts and present reader/player UI safely.

Recommended Flow

1. Ensure user is signed in.
2. Observe checkouts (`observeCheckouts`) for UI list updates.
3. Request missing checkout (`checkoutBook(with:)`) when needed.
4. Present using `openCheckout(_:presentedBy:customCover:)` on main actor.

Checkout + Open Example

```
func open(isbn: String, presenter: UIViewController) async {
   let result = await WeDoBooksFacade.shared.bookOperations.checkoutBook(with:
isbn)
    switch result {
    case .success(let checkout):
        do {
              try WeDoBooksFacade.shared.bookOperations.openCheckout(checkout,
presentedBy: presenter)
        } catch {
            print("Open failed: \(error)")
```

```
        }
    case .failure(let error):
        print("Checkout failed: \(error)")
    }
}
```

Notes

- Only one SDK book UI can be open at a time.
- If headless audiobook playback is active, `openCheckout` throws `OpenCheckoutError.aBookIsAlreadyOpen`.
- Ebook open path may show download/progress UI before reader when local file is missing.

Cross References

- DocC: BookOperations.md

Offline and Storage

Goal

Support deterministic download/remove/status behavior for offline book access.

Core APIs

- `storageOperations.download(book:)`
- `storageOperations.downloadStatus(book:)`
- `storageOperations.isBookDownloaded(isbn:)`
- `storageOperations.getAllDownloadedBooks()`
- `storageOperations.removeDownload(isbn:)`
- `storageOperations.onError`
- `storageOperations.downloadStatuses`

Download Example

```
func startOffline(_ checkout: Checkout) {
    do {
        try WeDoBooksFacade.shared.storageOperations.download(book: checkout)
    } catch {
        print("Download request failed early: \(error)")
    }
}
```

Observe Async Errors

```
private var cancellables: Set<AnyCancellable> = []

func observeStorageErrors() {
    WeDoBooksFacade.shared.storageOperations.onError
        .sink { error in
            print("Storage async error: \(error)")
        }
```

```
        .store(in: &cancellables)
}
```

Observe Download Statuses

```
func observeDownloadStatuses() {
    WeDoBooksFacade.shared.storageOperations.downloadStatuses
        .sink { statusesByISBN in
            print("Known download statuses: \(statusesByISBN)")
        }
        .store(in: &cancellables)
}
```

Notes

- Download failures can be synchronous (`throw`) or asynchronous (`onError`).
- Status is exposed as `StorageDownloadStatus`: `initializing`, `downloading(progress:)`, `downloaded`, `notDownloaded`, `failure`, or `cancel`.
- `initializing`, `failure`, and `cancel` are public enum cases. Integrations with exhaustive `switch` statements over `StorageDownloadStatus` must handle them.
- `downloadStatuses` emits the current snapshot immediately for the current user's persisted downloads plus active/known download states, then emits whenever a known book's state or progress changes.
- `downloadStatuses` is scoped to the current signed-in user. Switching users resets in-session download tracker state before the new user's snapshot is emitted.
- User sign-out clears relevant local storage state via facade delegate flow.

Cross References

- DocC: StorageOperations.md

Customization

Goal

Adjust SDK behavior and visual integration without forking SDK internals.

Available Areas

- `configuration`: feature toggles and persisted behavior flags
- `styling`: light/dark theme override
- `localization`: language selection and string overrides
- `images`: icon and reader image overrides
- `events`: host-side reactions to SDK UI actions

Configuration Example

```
let facade = WeDoBooksFacade.shared
facade.configuration.showFinishEbookButton = true
facade.configuration.showFinishAudiobookButton = true
facade.configuration.showAboutAudioBookButton = true
```

Localization Override Example

```
let overrides: [WeDoBooksFacade.Localization.LocalizationKeys:
[WeDoBooksFacade.Localization.Language: String]] = [
    .buttonSave: [.english: "Save"],
    .playerPlaybackRateSpeed: [.english: "Speed"]
]

WeDoBooksFacade.shared.localization.setCustomLocalizations(overrides)
```

Events Example

```
private var cancellables: Set<AnyCancellable> = []

func observeEvents() {
    WeDoBooksFacade.shared.events.finishBookTapped
        .sink { isbn in
            print("Finish tapped for \(isbn)")
        }
        .store(in: &cancellables)
}
```

Cross References

- DocC: Configuration.md
- DocC: Styling.md
- DocC: Localization.md
- DocC: Images.md
- DocC: Events.md

Headless Audio and CarPlay

Goal

Use SDK audiobook runtime without presenting SDK player UI, and optionally integrate transport controls in host app surfaces.

Headless Flow

1. Ensure setup + sign-in is completed.
2. Load audiobook checkout with `headlessAudioPlayer.loadBook(...)`.
3. Use `play`, `pause`, `seek`, `setRate` controls.
4. Observe `onStateChange` and `onError` on main queue.

Headless Example

```
func startHeadless(_ checkout: Checkout) {
    do {
        try WeDoBooksFacade.shared.headlessAudioPlayer.loadBook(book: checkout,
startPosition: 0)
        try WeDoBooksFacade.shared.headlessAudioPlayer.play()
    } catch {
        print("Headless failed: \(error)")
```

```
    }
}
```

CarPlay Example

```
func wireNowPlaying() {
    _ = WeDoBooksFacade.shared.carPlay.nowPlayingInfo
        .sink { info in
            print("Now playing info changed: \(String(describing: info))")
        }
}
```

Notes

- Headless and SDK Player UI share the same underlying runtime.
- Runtime ownership collisions are surfaced as deterministic errors.
- CarPlay controls are non-throwing helpers over the shared runtime.

Cross References

- DocC: HeadlessAudioPlayer.md
- DocC: CarPlay.md

Error Handling and Troubleshooting

Goal

Provide quick diagnosis paths for common setup/auth/open/storage/headless failures.

Error Families

- `SetupError`: setup lifecycle and reader initialization.
- `SignInError`: auth and token/factory integration issues.
- `CheckoutError` / `OpenCheckoutError`: checkout/open/presentation failures.
- `StorageOperationError`: offline and download issues.
- `HeadlessAudiobookError`: headless runtime state and arbitration errors.

Troubleshooting Matrix

Symptom	Likely Cause	First Check
`setupAlreadyCalled`	setup executed more than once	app lifecycle + singleton setup location
`firebaseNotConfigured`	missing/invalid firebase app config	adapter factory + config plist path
`noUserSignedIn`	user-dependent API before sign-in	auth token flow and sign-in result handling
`aBookIsAlreadyOpen`	runtime ownership collision	existing SDK player/reader or headless playback
`downloadNotAllowedOnCellular`	cellular download policy	`configuration.allowEbookDownloadUsingMobil`

Diagnostics Checklist

1. Confirm `setup(...)` succeeded this launch.
2. Confirm expected backend mode (`streaming` or `library`).
3. Confirm current user id is non-nil before user-scoped APIs.
4. Confirm runtime ownership before open/headless transitions.
5. Confirm network/download policy assumptions.

Cross References

- DocC: ErrorHandling.md